

NETFLIX

SYSTEM ARCHITECTURE

A Comprehensive Reverse-Engineering Analysis
End-to-End Architecture: CDN, APIs, DRM, Streaming & Scalability

Classification: Research · Analytical Report
Based on: Traffic Captures · Netflix Tech Blog · APNIC · FreeBSD Case Studies
June 2026

Executive Summary

Netflix operates at massive scale, delivering tens of terabits per second of video globally via its proprietary Open Connect CDN. This report reverse-engineers Netflix's system design by synthesising traffic captures, public engineering blogs, and distributed-systems principles.

The analysis covers the end-to-end architecture: from the client-side Single Page Application (Akira) and player (Cadmium), through dual API layers (legacy Falcor/Shakti and modern GraphQL), search and recommendation engines, to the detailed streaming pipeline incorporating FTL probes, DRM, and byte-range video delivery.

We explain how Netflix's custom CDN (Open Connect) caches and serves content close to ISPs, and how security layers (MSL, Widevine) are deeply integrated throughout. The content data model and microservice topology are outlined, alongside scalability and fault-tolerance strategies including multi-region deployment and eventual consistency.

SCALE AT A GLANCE

Netflix serves approximately 95% of all video traffic through its proprietary Open Connect CDN, operating at a reported peak throughput exceeding 100 Tb/s — a scale that no single commercial CDN currently matches.

1. Methodology

This analysis combines four complementary approaches to produce a rigorous reconstruction of Netflix's production architecture:

- **Traffic Capture Analysis:** Real Netflix web traffic was examined via a detailed teardown gist, exposing API calls, request/response payloads, streaming patterns, endpoint structures (GraphQL, Falcor, MSL, FTL), device headers (ESN, cookies), and the full sequence of events during playback.
- **Public Engineering Sources:** Netflix's own publications were surveyed, including Open Connect guides, the Netflix Tech Blog, and GitHub documentation, alongside third-party analyses from APNIC, the FreeBSD Foundation, and Designing Data-Intensive Applications (DDIA).
- **System-Design Principles:** Distributed systems best practices from DDIA and CDN research were applied to interpret architectural choices. For instance, CAP-theorem implications guided assumptions about consistency vs. availability trade-offs where Netflix's exact strategy was undocumented.

- **Reverse-Engineering Inference:** When specifics were not published, plausible designs were inferred from observed behavior — including decoding URL tokens, parsing cache headers to understand streaming authentication, and analysing JSON keys to reconstruct the content database schema. All assumptions are noted explicitly.

2. Architecture Overview

Netflix's system consists of client applications (web SPA, mobile apps, smart TVs), backend services hosted on AWS, and the Open Connect CDN for video delivery. Client requests flow to application services handling authentication, recommendations, and search — which ultimately yield signed URLs on Open Connect Appliances (OCAs) for streaming.

Core Components

- **Akira SPA & Cadmium Player:** The browser UI (React on Netflix's Akira framework) and video player (Cadmium) run client-side. Cadmium negotiates DRM via Widevine and manages adaptive bitrate streaming.
- **API Gateway:** An Envoy-based edge proxy in AWS front-ends all service calls, applying authentication and injecting B3 distributed-tracing headers.
- **Auth (Passport):** Manages login, session cookies, and profile selection. Issues long-lived NetflixId cookies (1-year TTL) and short-lived SecureNetflixId tokens.
- **Catalog Service:** Stores all content metadata — titles, cast, seasons — and exposes it via both GraphQL and Falcor APIs backed by the same underlying databases.
- **Search (Pinot):** Full-text search engine (Apache Pinot) handles queries and typeahead suggestions using capability-negotiation GraphQL queries to tailor results to client capabilities.
- **Recommendation Engine:** Generates personalised rows (e.g. 'Top 10', 'Because You Watched X') based on collaborative filtering, content embeddings, and user signals.
- **Play API / Licensed Manifest Service:** Generates DRM-protected playback manifests and signs OCA streaming URLs. Runs prefetch on the browse page to eliminate play-button latency.
- **Open Connect CDN:** Netflix's proprietary CDN. The control plane receives health metrics and available-asset reports from OCAs and steers playback requests to the optimal appliance.
- **Event Logging (Ichnaea):** High-throughput infrastructure collecting user events and device telemetry for analytics, personalisation, and observability.

ARCHITECTURAL PHILOSOPHY

The architecture prioritises Availability and Partition Tolerance (AP) under the CAP theorem. The vast majority of video data is served from local OCAs, reducing backbone load and

 keeping latency minimal for end users.

3. Client Architecture

The Netflix client comprises two distinct subsystems: the SPA shell (Akira) handling all UI concerns, and the video player (Cadmium) managing all media concerns. Each has clearly delineated responsibilities.

Akira UI Framework

Netflix's web application is a React SPA built on the Hawkins design system (v5.13 observed). The shell is server-rendered for initial HTML, then hydrates on the client. Key behaviours include:

- Client capabilities (supported entity types: VIDEO, GAME, GENERIC_CONTAINER) are embedded in every API query via a JSON pageCapabilities object.
- The x-netflix.context.ui-flavor: akira and x-netflix.clienttype: akira headers confirm the internal framework name.
- Profile management and authentication state are maintained through a cookie chain (NetflixId + SecureNetflixId).

Cadmium Video Player

The Cadmium player (cadmium-playercore) is loaded asynchronously and handles all streaming concerns:

- **DRM Negotiation:** Interfaces with Netflix's MSL/Play API to obtain Widevine licenses on Chrome, confirmed by the ?drmSystem=widevine flag in API calls.
- **Adaptive Streaming:** Requests video segments via byte-range HTTP requests. Notably, Netflix does not use DASH manifests — it operates its own byte-range protocol.
- **Subtitle Rendering:** Renders TTML subtitles using a custom format with 10-million-ticks-per-second timing for frame-accurate synchronisation.
- **Quality Monitoring:** Reports playback QoS metrics, manages client-side ABR decisions, and handles Chromecast integration (cast_sender.js is always loaded).

Device Capability Negotiation

Netflix tailors content delivery to client capabilities through query parameters embedded in every API request:

```
webp=true           // Client supports WebP images
drmSystem=widevine  // DRM system in use
isTop10Supported=true // UI can render Top 10 badges
hasVideoMerchInJaw=true // Feature flag for UI element
x-netflix.esn: NFCDCH-MC-* // Device Electronic Serial Number
```

4. API Layer Analysis

Netflix maintains two parallel browse API layers simultaneously: a legacy Falcor-based system (Shakti) and a modern GraphQL API. This dual-system design reflects an in-progress migration, enabling zero-downtime transition. Both APIs use persisted queries — a query ID rather than raw query text — keeping request payloads small.

Aspect	GraphQL API (New)	Falcor / Shakti API (Legacy)
Endpoint	POST /graphql (web.prod.cloud.netflix.com)	POST /nq/website/memberapi/release/pathEvaluator
Query Format	Persisted query ID (UUID)	JSON paths (Falcor JSON-Graph)
Payload Size	Small: ID + variables only	Larger: path expressions
Response Format	Typed JSON (Show/Movie/Episode)	JSON Graph with \$type atoms
Capabilities	Implicit in query structure	URL params (webp, drmSystem, etc.)
Status	Active — new features	Legacy — phasing out

GraphQL API Deep Dive

The GraphQL layer handles browsing, search, and hover previews. Observed persisted queries include DetailModal (full title metadata), MiniModalQuery (hover-preview data, ID: ec38bae4-2d3f-4f51-840e-f60ad596e3d1), and SearchPageQueryResults. A DetailModal response contains:

- title, synopsis, contentAdvisory (certification, maturity level, reasons)
- cast[], creators[], directors[] — arrays of Person objects with personId and name
- genreTags[], moodTags[] — typed taxonomy objects
- seasons.totalCount, currentEpisode (videoId, season, episode, runtimeSec)
- similars[] — related video IDs for 'More Like This' rows
- playbackBadges[] — e.g. VIDEO_HD, AUDIO_FIVE_DOT_ONE

5. Search Infrastructure

Netflix's search engine is built on Apache Pinot — a distributed OLAP engine also used by LinkedIn. Clients issue GraphQL `SearchPageQueryResults` queries including the search term and detailed `pageCapabilities`, allowing the server to tailor results to what the client can actually render.

CAPABILITY NEGOTIATION

Netflix's capability negotiation pattern allows new content types (e.g. cloud games) to be introduced without breaking older clients. The client declares exactly what entity types it can display; the server only returns compatible results.

Search Response Structure

A search request returns two main sections:

- **Typeahead Suggestions:** Short 'more to explore' list including collections and titles (e.g. `Collection:81137441` for a franchise group).
- **Search Results Gallery:** A grid of up to 48 matching titles with personalised `contextualArtwork` per query context.
- **Cache Expiry:** Results include an expires timestamp (observed ~8 hours ahead), indicating client-side TTL before a refetch is required.

Component	Netflix (Pinot-based)	Open-Source Alternative
Search Engine	Apache Pinot (distributed OLAP)	Elasticsearch, Apache Solr, Typesense
Query Interface	GraphQL with capability negotiation	REST or GraphQL over search core
Personalisation	Integrates user context per query	Custom layer (not in OSS core)
Suggestions	Integrated typeahead via Pinot	Dedicated fuzzy-search service
Caching	Client-side TTL via expires field	Cache-Control headers

6. Recommendation Infrastructure

Netflix's recommendation system assembles the personalised rows on the homepage. The pipeline follows a multi-stage funnel from signal collection through candidate generation to final ranking.

Pipeline Stages

- **Signal Collection:** User interactions (view history, ratings, search terms, completion rate, time-of-day patterns) are continuously logged via Ichnaea.
- **Candidate Generation:** Multiple methods run in parallel: collaborative filtering (latent factor models / matrix factorisation), content-based matching (genres, tags, actors), editorial curation (Top 10 global titles), and new-release promotions.
- **Feature Engineering:** For each candidate, features are computed: user-item affinity scores, novelty, completion rate, embedding similarity (user-vector · item-vector).
- **Ranking:** A learning-to-rank model (typically a neural network or gradient-boosted trees) scores candidates per user, incorporating business rules such as catalogue availability and content diversity constraints.
- **Result Assembly:** Top-ranked titles are slotted into fixed-size rows. The Lolomo (List of Lists of Movies) layout structure tracks each item's row and column position via a tctx tracking context.

7. Playback Pipeline

The video playback flow involves five distinct phases, each serving a specific purpose in reducing startup latency and ensuring secure, high-quality delivery.

Phase 1: FTL Network Probing

Before any video data is requested, Cadmium probes multiple CDN endpoints to determine the lowest-latency path. The `/api/ftl/probe` call returns a set of candidate URLs:

- A DNS-based resolver (ifurxi7eqx...nflxso.net)
- An anycast address for geographic routing
- An IX-specific URL (e.g. uswest-2.ftl.nflxvideo.net)
- An AWS origin fallback

The client races probes to each URL simultaneously, measuring round-trip latency. Probe pulses continue at intervals (0ms, 2000ms, 7000ms) throughout playback to adapt to changing network conditions.

Phase 2: MSL Token Provisioning (ALE)

The client establishes a secure session via Netflix's proprietary Message Security Layer (MSL) protocol. A POST to `/nq/msl_v1/nrdjs/...?reqName=aleProvision` carries an entirely encrypted

payload (headers and body). The response yields an encrypted token bundle containing sessiondata, expiry timestamps, and cryptographic counters.

Phase 3: Licensed Manifest (DRM)

With an MSL session established, the client requests the full DRM license and stream info in a single MSL-encrypted exchange. Critically, this request uses `reqName=prefetch/licensedManifest` — the manifest is fetched proactively while the user is still browsing, before they press play. The encrypted response contains:

- Widevine DRM license blob for the browser CDM
- Signed OCA streaming URLs (video and audio)
- Codec and resolution selection for the detected device
- Subtitle manifest URLs (separate TTML files)

PREFETCH OPTIMISATION

The prefetch strategy is one of Netflix's most impactful latency optimisations. By fetching the DRM manifest and license while the user browses, playback can begin in milliseconds when they press play — the expensive cryptographic operations are already complete.

Phase 4: Byte-Range Streaming

Cadmium fetches video segments via HTTP GET with Range headers to the selected OCA URL. The OCA server runs FreeBSD with NGINX, optimised for high-throughput file serving. Segment characteristics observed:

- Video segments: approximately 300–600 KB per request
- Audio segments: approximately 25–50 KB per request
- Video and audio fetched in parallel, interleaved
- URL parameters: `o=1` (offset flag), `v=31` (protocol version), `e=<expiry>` (~12h Unix timestamp), `t=<signed-token>` (prevents hotlinking)

Phase 5: Subtitle Delivery

Subtitle files (TTML format) are fetched separately from the OCA in parallel with video. Netflix's TTML uses 10-million-ticks-per-second timing resolution for frame-accurate synchronisation. Because subtitles are external files rather than embedded tracks, language switching is instantaneous.

8. Open Connect CDN Deep Dive

Netflix serves virtually all video traffic through its custom Open Connect CDN. Hardware appliances (OCAs) are deployed in strategic locations globally, combined with a centralised AWS control plane. The design principle is simple but powerful: move content as close to viewers as physically possible.

Deployment Models

Model	Description	Routing	Priority
IX Peering	OCAs at Internet Exchange Points. Netflix peers settlement-free with any ISP (AS2906).	BGP via IXP fabric; AS path length 2	Secondary (fallback)
Embedded Appliances	Netflix provides free OCA hardware to qualifying ISPs for installation inside their network (AS40027).	BGP direct; AS path length 1	Primary (preferred)

BGP routing rules naturally prefer embedded OCAs over IX OCAs: the shorter AS path (length 1 vs 2) wins automatically. Netflix covers all hardware costs; the ISP provides rack space, power, and a BGP session.

Cache Control & Steering

- **Cache Control Service (CCS):** OCAs report their health, routing tables, and which content chunks they hold. CCS aggregates this in AWS and provides the steering service with a live inventory.
- **Steering Service:** Consults CCS and the user's resolved IP location to select the optimal OCA(s). Generates signed URLs for the chosen appliance(s). Steering logic criteria, in priority order: longest-prefix match, shortest AS path, lowest MED, then geography.
- **Fill Windows:** Content is pushed to OCAs during off-peak hours. Popular titles are proactively distributed; long-tail content may require origin fetch on first request.
- **Directed Caching:** Each OCA is a directed cache — it only accepts requests for IP prefixes it has learned via BGP. This prevents cache pollution and ensures OCAs only serve traffic from their designated network segment.

9. DRM & Security Architecture

Netflix employs a multi-layered security model integrating transport encryption, custom application-level security, and industry-standard DRM.

Message Security Layer (MSL)

MSL is Netflix's proprietary end-to-end encryption protocol for API calls. Any request with Content-Encoding: msl_v1 has its entire payload — headers and body — encrypted client-side. MSL provides:

- Session tokens (mastertoken) for authenticated request binding
- Payload confidentiality via Base64-encoded symmetric encryption
- Replay protection via cryptographic counters
- Used for both ALE provisioning and licensed manifest exchanges

Widevine DRM

Netflix uses Google's Widevine Content Decryption Module (CDM) on Chrome (and PlayReady on other platforms). The CDM operates within the browser's sandboxed environment. The licensed manifest response delivers a Widevine license blob, which is passed to the CDM to decrypt stream segments.

Tokenised Streaming URLs

Every OCA streaming URL includes two security parameters:

- `e=<expiry>`: Unix timestamp approximately 12 hours in the future. After this point, the URL is cryptographically invalid and will be rejected by the OCA.
- `t=<signed-token>`: A signed JWT encoding session identity (MSL key info). Prevents URL sharing and hotlinking.

Session Cookie Chain

Authentication uses a two-cookie chain: a persistent `NetflixId` (1-year TTL) and a short-lived `SecureNetflixId`. Both use `HttpOnly`, `Secure`, and `SameSite` attributes. The user profile GUID is encoded in cookies and explicitly echoed in request headers (`x-netflix.request.client.user.guid`) for every API call.

10. Content Data Model

Netflix's content catalog follows a typed entity-relationship model, reconstructed from GraphQL response schemas. The base entity is `Video`, with `Show` and `Movie` as subtypes.

Entity	Key Fields	Notes
Video (base)	videoId, title, isPlayable, unifiedEntityId	Root type; links to cast, genres, supplementals
Show (extends Video)	numSeasons, seasons[], episodes[]	__typename: 'Show'; includes flat episode list
Movie (extends Video)	runtimeSec, videoId	__typename: 'Movie'; no seasons
Episode	number, title, runtimeSec, watchStatus	Belongs to Season; tracks play state
Season	number, episodes[]	Belongs to Show
Person	personId, name	Cast, directors, creators
Genre	genreId, name	e.g. 'Romantic TV Dramas', 'Period Pieces'
Supplemental	computeId, offset, durationSec	Trailers and promos
Lolomo Row	heading, type, videos[]	Browse layout — List of Lists of Movies

11. Scalability & Fault Tolerance

Netflix's architecture is engineered for global scale and aggressive fault tolerance. Several complementary strategies work in concert.

Microservices Architecture

Netflix was an early pioneer of the microservices pattern. Each functional component — auth, catalog, play, search, recommendations — is a separate, independently deployable service. This enables independent scaling: the Play API can absorb peak traffic spikes without impacting the recommendation engine, and vice versa.

Chaos Engineering

Netflix's famous Chaos Monkey randomly terminates production service instances to verify that the system remains functional under arbitrary failure. Every service is designed to expect failure of any dependency and to degrade gracefully rather than cascade. This philosophy has proven highly effective — Netflix maintains extremely high availability despite running on shared cloud infrastructure.

CAP Theorem Trade-offs

Netflix explicitly favours Availability and Partition Tolerance (AP) over strong Consistency in most services. Key examples:

- Browse rows and recommendations tolerate eventual consistency — a new title may not appear in all clients' recommendations immediately.
- Watch history and continue-watching data uses relaxed consistency — a bookmark may be briefly stale but never lost.
- Account changes and DRM licensing likely require stronger consistency (CP behaviour) given the financial and security implications.

Caching Tiers

- **Edge Tier:** Open Connect Appliances cache video segments close to end users, handling ~95% of all video traffic.
- **Application Tier:** EVCache (multi-cluster Memcached) caches user profiles, search indices, and popular catalog data.
- **Session Tier:** Sparta caches API tokens and session state, reducing load on the Passport auth service.
- **Static Assets:** UI images and posters are served via third-party CDNs (Akamai) rather than Open Connect.

12. Recreating Netflix: Open-Source Stack

To implement a Netflix-like system as a minimal viable prototype, the following services and open-source alternatives map to each Netflix component:

Netflix Component	Function	Open-Source Alternative
Auth (Passport)	Session, profile, ESN management	Keycloak, Dex (OAuth/OIDC)
Catalog / Metadata DB	Content titles, cast, genres	PostgreSQL + Apollo GraphQL Server
Search (Pinot)	Full-text search + typeahead	Apache Pinot (OSS), Elasticsearch
Recommendation Engine	Candidate generation + ranking	Apache Spark MLlib, LensKit (CF)
Play API / DRM	Manifest generation, license issuance	Shaka Packager + custom service

Netflix Component	Function	Open-Source Alternative
Open Connect CDN	Byte-range video delivery	CloudFront / Cloudflare + NGINX
Image API (dnm)	On-the-fly poster resizing	Thumbor, imgproxy
Subtitle Service	TTML/WebVTT subtitle delivery	Object store (S3) + WebVTT
API Gateway	Envoy/Zuul with B3 tracing	Kong, Traefik, or Envoy OSS
Event Logging (Ichnaea)	User event collection	Kafka + Apache Druid / ClickHouse
Monitoring & Tracing	Health checks, distributed traces	Prometheus + Grafana + Jaeger
Client (Akira + Cadmium)	SPA shell + video player	React.js + Shaka Player / Video.js

13. Key Findings & Engineering Lessons

The analysis yielded several non-obvious insights with broad applicability to high-scale system design.

- **Dual API Migration Strategy:** Running Falcor/Shakti and GraphQL in parallel allowed Netflix to migrate APIs without any downtime or client breakage. New features are built on GraphQL while legacy routes continue to serve older clients — a textbook blue-green API migration.
- **Prefetch-Driven Instant Playback:** The DRM manifest and first video segments are fetched while the user browses, not when they click play. This aggressive prefetch strategy makes playback feel instantaneous — the expensive cryptographic operations complete during idle time.
- **Capability Negotiation over Versioning:** Rather than maintaining multiple API versions for different client capabilities, Netflix embeds a capability declaration in every request. The server tailors its response to what the client can actually render, enabling seamless feature rollouts without client coordination.
- **Directed CDN Caching:** By routing OCA traffic exclusively via BGP-announced prefixes, Netflix ensures each appliance only serves a defined set of users. This prevents cache pollution and delivers very high cache hit rates for popular content in each network segment.
- **CDN Failover at the Streaming Level:** During playback, the Cadmium player transparently switched between multiple OCA nodes (c345, c310, c314) at the same San Jose IX. Redundancy is implemented at the edge, not just the origin, ensuring seamless recovery from appliance failures.
- **B3 Tracing as a Standard:** Every API response carries an x-b3-traceid header, enabling end-to-end request tracing across all microservices. This observability instrumentation is essential for diagnosing failures in a system of this complexity.

- **Feature Flags in Query Parameters:** Dozens of URL parameters (hasVideoMerchInBob, hasVideoMerchInJaw, etc.) act as per-session feature flags, allowing Netflix to enable or disable UI features dynamically without code deployments.

References

The following sources informed this analysis:

- Tandon, N. Netflix Content Distribution through Open Connect. APNIC Blog, June 2018.
- Netflix Open Connect: Official documentation and ISP partner guides. openconnect.netflix.com
- Wallace, G. Open Connect Case Study. FreeBSD Foundation Journal, Jan/Feb 2021.
- Kleppmann, M. Designing Data-Intensive Applications. O'Reilly Media (Chapter 9: Consistency and Consensus).
- sshh12. Netflix's web architecture reverse-engineered from live network traffic. GitHub Gist, 2025.
- Netflix Tech Blog. Migrating Netflix to GraphQL Safely. netflixtechblog.com
- Netflix Tech Blog. Microservices at Netflix. netflixtechblog.com
- System Design Primer. github.com/donnemartin/system-design-primer
- Lebes, G. Netflix System Design: Backend Architecture. DEV Community, dev.to

Appendix: Diagram Generation

The architectural diagrams referenced in this report can be reproduced using the following tooling:

Mermaid CLI

```
# Install Mermaid CLI
npm install -g @mermaid-js/mermaid-cli

# Render architecture diagram
mmdc -i architecture.mmd -o architecture.png

# Render sequence diagram (playback pipeline)
mmdc -i playback_sequence.mmd -o playback.png
```

GraphQL Schema Visualisation

```
# Visualise Netflix content schema from GraphQL introspection
npx graphqlviz https://web.prod.cloud.netflix.com/graphql > schema.dot
dot -Tpng schema.dot -o content_schema.png
```

Traffic Analysis

Technical details were derived by inspecting HTTP captures via browser DevTools and mitmproxy. To replicate the traffic analysis:

- Configure mitmproxy as a transparent proxy on the Netflix-connected device.
- Navigate to Netflix web (us-west-2 region recommended for best endpoint coverage).
- Capture and analyse GraphQL, Falcor, MSL, and FTL probe requests in the network log.
- Decode Base64-encoded MSL payloads to inspect token structure (note: payload content is encrypted).